Knowledge Neurons in Pretrained Transformers

Reproducing and Extending Fact-Editing Experiments in BERT and GPT-2

XAI Course Project Report

Hamza Azzouzi Jules Damidaux

April 2026

Abstract

This report studies the work of Dai et al. (2022) on *Knowledge Neurons in Pretrained Transformers*. The original paper proposes a method to locate specific neurons in feed-forward layers of BERT that store factual knowledge, and shows that manipulating these neurons can update or erase facts without any fine-tuning. We reimplement the core pipeline (integrated gradients attribution, neuron suppression and amplification, weight-level editing) and test it on three new scenarios with BERT: making the model believe that France's capital is Tokyo, that French people speak German, and that Einstein was born in Paris. While these edits succeed on the target fact, they damage related facts. We trace this problem to neuron overlap between related facts, propose a fix based on exclusive neurons, and quantify the trade-off through a λ-sweep. We also adapt the method to GPT-2 and compare how knowledge neurons differ between masked and autoregressive models. The code for our experiments is available at https://github.com/julesdamidaux/knowledge-neurons.

1 Introduction

Pretrained language models like BERT [2] absorb a huge amount of factual knowledge during training on large corpora such as Wikipedia. This raises an interesting question for explainable AI: *where exactly* does this knowledge live inside the model, and is it possible to change it in a targeted way?

Dai et al. [1] tackled this question by introducing the concept of **knowledge neurons**. These are individual neurons in the feed-forward network (FFN) intermediate layers whose activations correlate strongly with specific relational facts. Their paper makes three main contributions. First, they propose a knowledge attribution method based on integrated gradients that can pinpoint which FFN neurons encode a given fact. Second, they show through suppression and amplification experiments that these neurons have a direct causal effect on the model's factual predictions. Third, they demonstrate that modifying the weights connected to these neurons can update or erase facts, all without any fine-tuning.

In this project, we reimplement their pipeline on BERT-base-cased and test it with three new experiments. We also extend the method to GPT-2 to check whether it generalizes to autoregressive models.

2 Background

2.1 Transformer Architecture

A Transformer encoder [3] is made of L identical blocks, each with a self-attention module and a feed-forward network (FFN). The FFN takes the hidden state H from self-attention and computes:

$$\mathrm{FFN}(H) = \mathrm{gelu}(HW_1)\, W_2 \tag{1}$$

where $W_1 \in \mathbb{R}^{d \times d_m}$ and $W_2 \in \mathbb{R}^{d_m \times d}$ are two linear layers and d_m is the intermediate size (3072 for BERT-base).

2.2 FFN as Key-Value Memory

Geva et al. [4] observed that FFN layers behave like key-value memories. The first layer W_1 acts as *keys* that detect patterns in the input, while the second layer W_2 stores *values* containing the information to retrieve. The intermediate neurons after the GELU activation serve as memory coefficients that control how much each value vector contributes. This interpretation motivates the search for knowledge neurons: if the FFN is a memory, then specific neurons should be responsible for specific pieces of knowledge.

3 Methodology

3.1 Knowledge Attribution via Integrated Gradients

Given a cloze prompt like "*The capital of France is [MASK]*" with correct answer y^* = "Paris", we define the model output as $P_x(w_i^{(l)}) = p(y^* \mid x, w_i^{(l)})$, where $w_i^{(l)}$ is the activation of the i-th neuron in the l-th FFN layer. We measure each neuron's importance using **integrated gradients** [5]:

$$\mathrm{Attr}(w_i^{(l)}) = \overline{w}_i^{(l)} \int_0^1 \frac{\partial P_x(\alpha \overline{w}_i^{(l)})}{\partial w_i^{(l)}} \, d\alpha \tag{2}$$

where $\overline{w}_i^{(l)}$ is the actual activation value. We approximate this integral with $m = 20$ Riemann steps.

3.2 Knowledge Neuron Identification

For each fact, we use several paraphrased prompts (typically 4 or 5). For each prompt, we compute integrated gradients for all $L \times d_m$ neurons, keep only the **positive** attributions (the neurons that help predict the right answer), and retain those with a score of at least $0.2 \times \max(\text{attributions})$. We then keep neurons that appear in at least 30% of the prompts.

We found that keeping only positive attributions is important. When we initially kept all high-magnitude neurons regardless of sign, suppressing them would paradoxically *increase* the target probability, because we were also suppressing inhibitory neurons.

3.3 Knowledge Surgery

To change a fact from answer t to t', we modify the FFN output weights W_2 at each knowledge neuron position:

$$W_2[:, i] \leftarrow W_2[:, i] - \lambda_1 \hat{\mathbf{e}}_t + \lambda_2 \hat{\mathbf{e}}_{t'} \tag{3}$$

where $\hat{\mathbf{e}}_t$ and $\hat{\mathbf{e}}_{t'}$ are the L_2-normalized word embeddings of the old and new answers. The parameters λ_1 and λ_2 control how aggressively we remove the old knowledge and inject the new one.

4 Experimental Setup

We use BERT-base-cased (12 layers, 768 hidden dimensions, 3072 FFN intermediate size, 108M parameters) from HuggingFace Transformers. All experiments were run on a single NVIDIA RTX A5000 GPU. Identifying knowledge neurons for one fact takes about 20 seconds with 5 prompts and 20 integration steps.

We designed three experiments, each involving a fact that BERT knows well. Note that even for well-known facts, BERT never assigns 100% probability to the correct answer because it distributes

probability mass over plausible completions. For example, for "The capital of France is [MASK]", BERT gives 44% to Paris but also 9% to Lyon, 8% to Toulouse, etc. Similarly, "The capital of Japan is [MASK]" gets 61% for Tokyo but also 11% for Osaka and 9% for Kyoto.

Our three experiments are:

1. **Capital Swap**: make BERT say "The capital of France is Tokyo" instead of Paris.
2. **Language Confusion**: make BERT believe "The official language of France is German".
3. **Einstein Teleportation**: make BERT think "Albert Einstein was born in Paris".

For each experiment, we evaluate: knowledge neuron distribution across layers, the effect of suppression and amplification, edit success, and collateral damage on related facts.

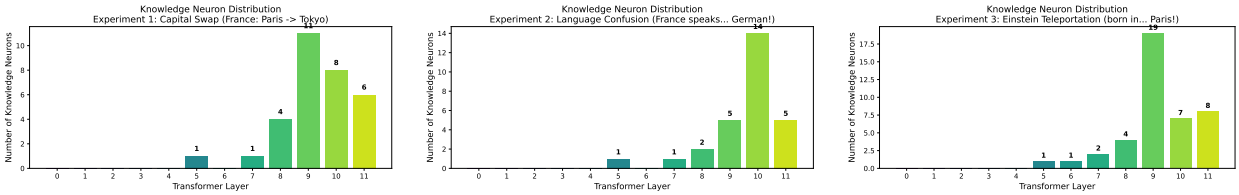
5 Results on BERT

5.1 Knowledge Neuron Identification

Table 1 shows how many knowledge neurons we found for each experiment. As shown in Figure 1, they consistently appear in the upper layers (8 to 11), matching the original paper's findings.

Table 1: Knowledge neurons identified per experiment.

Experiment	Total KN	Peak Layer
Capital Swap (Paris)	27	9
Language Confusion (French)	28	10
Einstein Teleport. (Germany)	42	9



(a) Capital Swap (b) Language Confusion (c) Einstein Teleportation

Figure 1: Distribution of knowledge neurons across layers for each experiment.

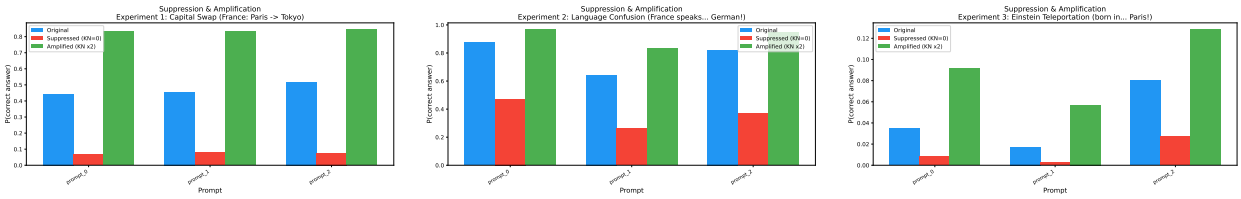
5.2 Suppression and Amplification

To validate that these neurons actually encode the target knowledge, we tested two interventions at inference time: setting their activations to zero (suppression) or doubling them (amplification).

Table 2: Effect of suppression and amplification on P(correct answer), averaged across prompts.

Experiment	Original	Suppressed	Amplified
Capital Swap (Paris)	0.4736	0.0734 (−84.5%)	0.8358 (+76.5%)
Language Confusion (French)	0.7803	0.3685 (−52.8%)	0.9157 (+17.3%)
Einstein Teleport. (Germany)	0.0440	0.0126 (−71.4%)	0.0921 (+109%)

The results are clear (Table 2 and Figure 2). Suppression consistently reduces the probability of the correct answer. For the capital swap, P(Paris) drops from 44% to 7%. After suppression, BERT predicts "Toulouse" instead, which is consistent with the suppressed neurons encoding the specific fact "Paris is the capital" rather than a generic "French city" association. Amplification has the expected opposite effect, boosting confidence substantially.



(a) Capital Swap (b) Language Confusion (c) Einstein Teleportation

Figure 2: Suppression (red) and amplification (green) compared to original (blue).

5.3 Knowledge Surgery

With the knowledge neurons identified and validated, we moved on to the main goal: editing facts. We applied Equation 3 with $\lambda_1 = 1$ and $\lambda_2 = 5$ to make BERT believe that France's capital is Tokyo, that the language of France is German, and that Einstein was born in Paris.

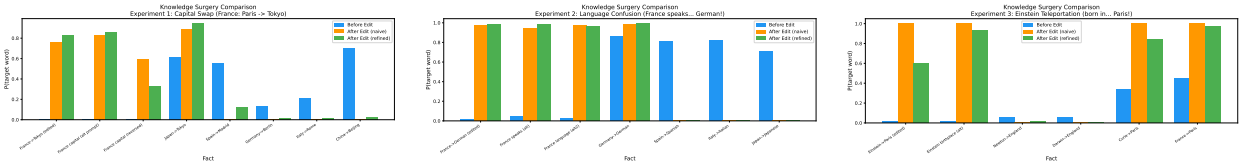
At first glance, the results looked very promising (Table 3). BERT goes from giving Tokyo a 0.01% chance of being France's capital to **78%**. For the language experiment, P(German) jumps from 1.5% to 98%. All three target facts become the model's top-1 prediction.

Table 3: Knowledge surgery results: P(new answer) after editing all knowledge neurons.

Experiment	**Before**	**After**	**Top-1?**
France → Tokyo	0.0001	0.7811	✓
France → German	0.0148	0.9770	✓
Einstein → Paris	0.0186	0.9994	✓

However, when we checked what happened to *other* capitals, the picture was much less encouraging. Figure 3 shows that after editing France's capital to Tokyo, BERT now predicts Tokyo for Spain, Germany, Italy, and China as well. The edit completely destroyed all capital-related knowledge, not just France's. The same pattern appeared for languages (all countries predict "German") and birthplaces (all scientists are "born in Paris").

This was unexpected. We had assumed that knowledge neurons for "France → Paris" would be different from those for "Spain → Madrid", but the edits clearly affected both.



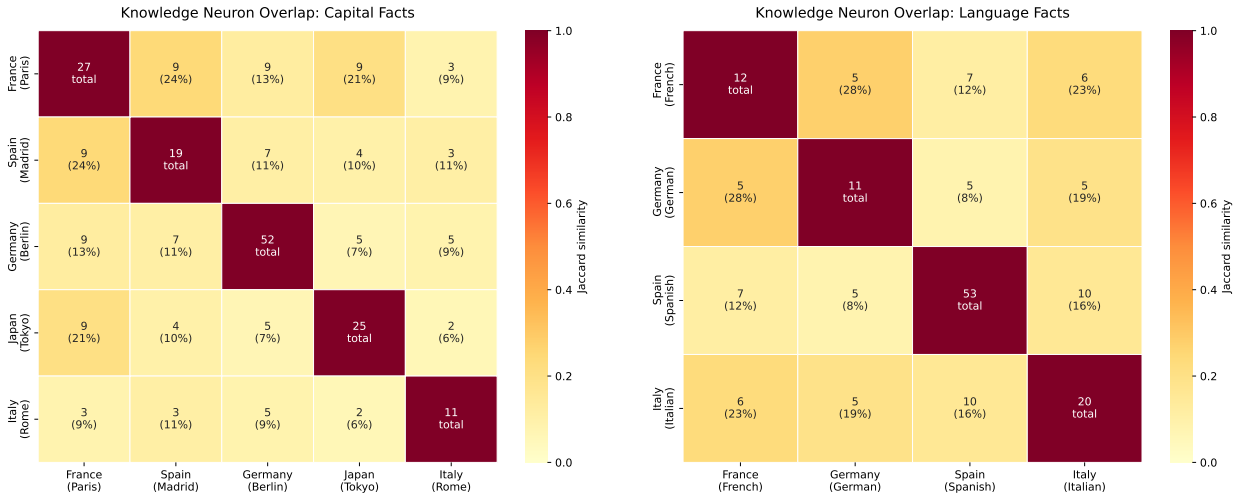
(a) Capital Swap (b) Language Confusion (c) Einstein Teleportation

Figure 3: Before (blue) vs. after editing all knowledge neurons (orange). Green bars show a mitigation strategy discussed in Section 5.5. The collateral damage on control facts is severe.

5.4 Investigating the Cause: Neuron Overlap Analysis

To understand why editing France's capital also destroyed Spain's, we ran a follow-up experiment. We identified the knowledge neurons for five capital facts *independently* (France, Spain, Germany, Japan, Italy) and then checked how much they overlap.

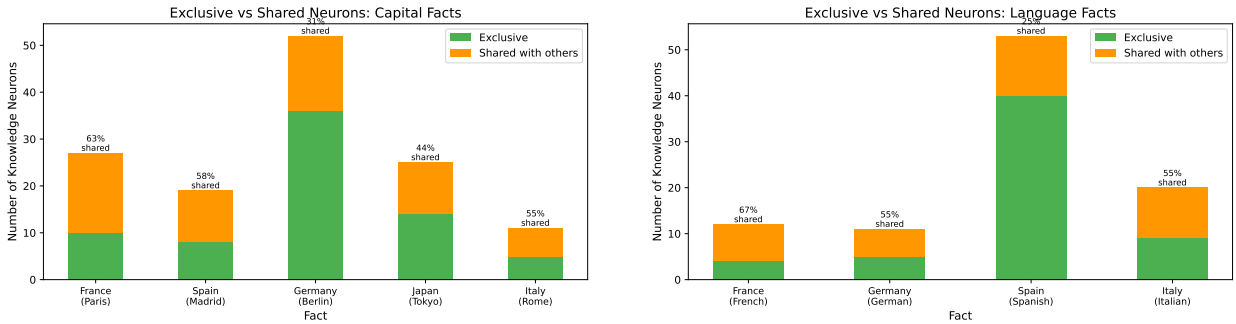
The results (Figure 4) reveal the mechanism. France and Spain share 9 knowledge neurons out of 27 and 19 respectively (24% Jaccard similarity). France and Japan also share 9. The neuron at layer 10, position 1122, is shared by **all five** capital facts, and three other neurons are shared by four out of five. Since these neurons activate for all five different countries, they encode the "capital of" relation itself rather than any specific country-capital pair. Editing them biases all capital predictions at once.



(a) Capital facts (b) Language facts

Figure 4: Pairwise knowledge neuron overlap. Each cell shows the number of shared neurons and the Jaccard similarity. The off-diagonal overlap directly explains the collateral damage.

Figure 5 quantifies this for each fact. For France's capital, 63% of its knowledge neurons are shared with at least one other capital fact. The picture is the same for languages: four neurons in layer 10 are shared by all four language facts. This means that knowledge neurons encode a mix of *relation-level* information ("capital of", shared) and *entity-level* information ("France-Paris", exclusive). Editing the shared neurons inevitably disrupts all facts of the same relation.



(a) Capital facts (b) Language facts

Figure 5: Exclusive (green) vs. shared (orange) neurons for each fact.

5.5 A Fix: Editing Only Exclusive Neurons

The overlap analysis pointed to a natural solution: if the shared neurons are the problem, what if we only edit the *exclusive* ones? For each fact, we can remove the neurons that also appear in related facts and keep only those specific to the target. For France's capital, this reduces the set from 27 to 10 neurons.

Figure 6 compares three strategies for the capital swap: editing all 27 neurons, editing 10 exclusive neurons (filtered against 3 countries), and editing 10 exclusive neurons (filtered against all 5 countries). In this case, filtering against 3 or 5 countries yields the same exclusive set, which means 3 related facts are enough to identify the shared neurons for this relation.

The results (Table 4) show a clear trade-off. Exclusive neurons preserve control facts significantly better (72% vs. 41%), and the improvement on Spain is dramatic: P(Madrid) goes from near zero to 0.48 (vs. 0.55 baseline, only 13% damage). The cost is a moderate reduction in edit strength (0.69 vs. 0.82), but the new answer is still the top-1 prediction.

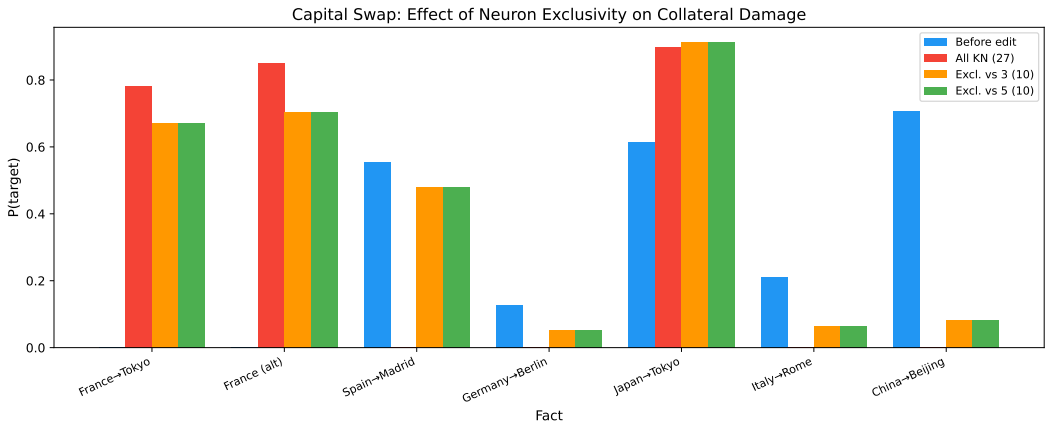


Figure 6: Effect of neuron exclusivity on collateral damage. Editing only exclusive neurons (orange/green) preserves Spain's capital much better than editing all neurons (red), while still successfully changing France's capital to Tokyo.

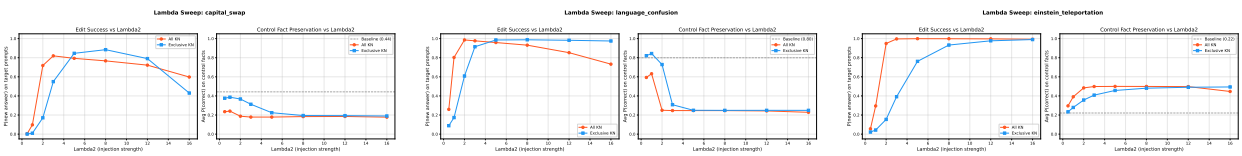
Table 4: Editing all neurons vs. exclusive neurons only for the capital swap.

	All KN (27)	**Exclusive (10)**
Edit success (avg P(Tokyo) on France)	0.815	0.688
Control preservation (avg P(correct) on others)	40.6%	71.9%
Spain → Madrid (P after edit)	0.0001	0.479

However, facts not included in the filter (like China) still suffer damage (P(Beijing) drops from 0.71 to 0.08). The filter only removes neurons that are *known* to be shared, so there may be additional shared neurons that our filter does not catch because we did not test against those specific facts.

5.6 Lambda Sweep: Tuning the Edit Strength

Another way to reduce collateral damage is to lower λ_2, the injection strength. We swept λ_2 from 0.5 to 16 while keeping $\lambda_1 = 1$ (Figure 7).



(a) Capital Swap (b) Language Confusion (c) Einstein Teleportation

Figure 7: Lambda sweep. Left panels: edit success vs. λ_2. Right panels: control fact preservation. Exclusive neurons (blue) preserve control facts better at low λ_2.

There is a sweet spot around $\lambda_2 \approx 3$ to 8 where the edit is strong but not yet degraded. Going beyond that hurts the edit itself: the modified weight vectors become so large that they dominate the FFN output regardless of input. The exclusive neuron approach buys some extra room at moderate λ_2 values but both strategies converge at high λ_2.

No configuration achieves both high edit success and full control preservation, confirming that weight-level surgery faces an inherent precision limit.

6 Extension to GPT-2

To test whether this approach works beyond masked language models, we adapted the pipeline to GPT-2 (124M parameters, 12 layers, same intermediate size of 3072 as BERT-base).

6.1 Adaptation

Instead of predicting a [MASK] token, we predict the *next* token after the prompt. So "Tokyo is the capital of" should predict " Japan" as the next word. The target position becomes the last token in the input, and the rest of the pipeline (integrated gradients, suppression, editing) is unchanged. The only technical difference is that GPT-2 uses `Conv1D` layers with transposed weights, so the value matrix W_2 is accessed as `c_proj.weight` with shape $(d_m \times d)$.

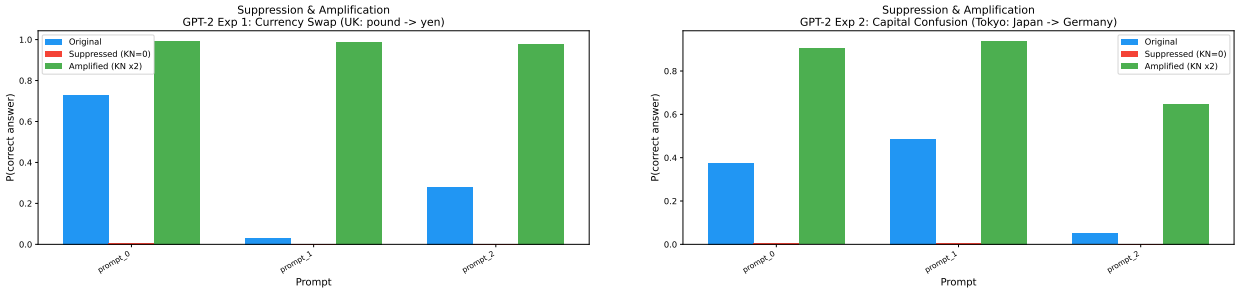
6.2 Experiments and Results

We ran two experiments: swapping the UK's currency from pound to yen, and changing Tokyo's country from Japan to Germany.

Table 5: GPT-2 results ($\lambda_1 = 1, \lambda_2 = 5$).

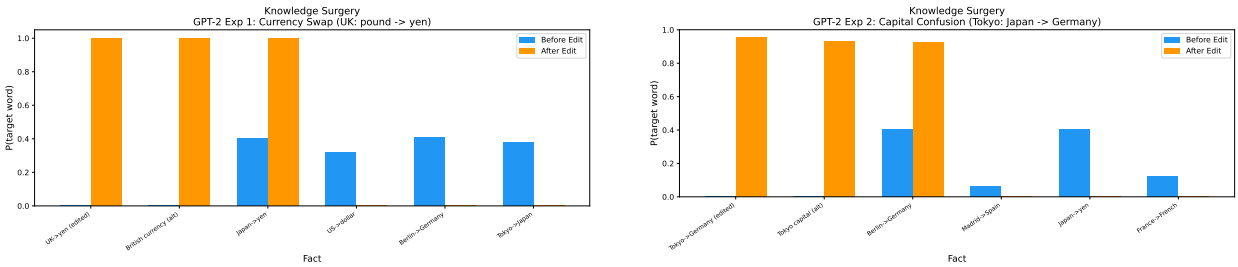
Experiment	KN found	Supp. effect	P(new) before	P(new) after
Currency: pound → yen	75	−99.3%	0.0005	1.0000
Capital: Japan → Germany	50	−98.6%	0.0004	0.9530

The method works on GPT-2 too, and in some ways even better (Table 5). Suppression almost completely eliminates the correct prediction (−99% vs. −85% for BERT), and the edits achieve near-perfect success. But the collateral damage is also worse. This is consistent with the fact that the method identifies more neurons for GPT-2 (50 to 75 vs. 27 to 42 for BERT): more edited neurons means a larger footprint in the weight space, affecting more unrelated inputs.



(a) Currency Swap (b) Capital Confusion

Figure 8: GPT-2 suppression and amplification. Effects are stronger than in BERT.



(a) Currency Swap (b) Capital Confusion

Figure 9: GPT-2 knowledge surgery. The edits succeed but collateral damage is severe.

6.3 Comparing BERT and GPT-2

Figure 10 summarizes the comparison. Three differences stand out. First, GPT-2 identifies more knowledge neurons per fact (50 to 75 vs. 27 to 42 for BERT). Second, the layer distribution is

very different: BERT concentrates its knowledge neurons in the top layers (8 to 11), while GPT-2 spreads them across all layers (0 to 11). We do not have a definitive explanation for this, though one possibility is that BERT can rely on bidirectional context to defer factual recall to later layers, whereas GPT-2 only sees left context. Third, suppression has a bigger effect in GPT-2 (-98% vs. -84%).

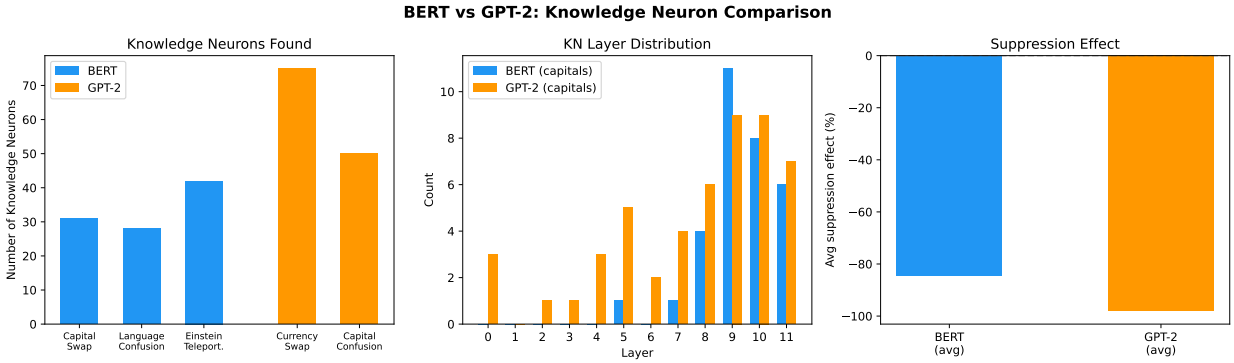


Figure 10: BERT vs. GPT-2: KN count (left), layer distribution (center), suppression effect (right).

7 Conclusion

We reproduced and extended the Knowledge Neurons framework through three experiments on BERT and two on GPT-2. Our results are consistent with Dai et al.: knowledge neurons can be reliably identified with integrated gradients and manipulated to alter factual predictions. The extension to GPT-2 confirms the method generalizes beyond masked language models.

Our main finding is that knowledge neurons encode a mix of *relation-level* information (shared across facts like "capital of France" and "capital of Spain") and *entity-level* information (specific to a single fact). The shared neurons are the direct cause of collateral damage during editing: one neuron at layer 10, position 1122 is active for all five capital facts we tested. Editing only the exclusive neurons preserves 72% of control facts compared to 41% when editing all neurons, at a moderate cost in edit strength (P(Tokyo) = 0.69 vs. 0.82).

Several limitations remain. Our method only handles single-token answers, we only tested on small models ($\sim$120M parameters), and the exclusive neuron filter only protects against facts it has been checked against (China's capital still suffers damage). More sophisticated methods like ROME [7], which uses rank-one updates with explicit locality constraints, were designed to address these precision issues.

References

[1] Damai Dai, Li Dong, Yaru Hao, Zhifang Sui, Baobao Chang, and Furu Wei. Knowledge neurons in pretrained transformers. In *Proc. ACL*, 2022.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proc. NAACL-HLT*, 2019.

[3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proc. NeurIPS*, 2017.

[4] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proc. EMNLP*, 2021.

[5] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *Proc. ICML*, 2017.

[6] Fabio Petroni, Tim Rocktäschel, Sebastian Riedel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, and Alexander Miller. Language models as knowledge bases? In *Proc. EMNLP-IJCNLP*, 2019.

[7] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Proc. NeurIPS*, 2022.